

RISC-V Custom Hardware Acceleration using Batch Normalization (bnorm)

Project Documentation

Group 10 – Batch Normalization

Compiler, Assembler and Simulator Co-Design using RISC-V

1. Introduction

This project demonstrates complete hardware-software co-design by integrating a custom Batch Normalization instruction called *'bnorm'* into the RISC-V ecosystem. The objective of the project is to allow a standard mathematical Batch Normalization equation written in C code to automatically compile into a dedicated hardware instruction.

Instead of executing multiple generic CPU instructions such as subtraction, division, multiplication and square root separately, the compiler recognizes the Batch Normalization computation pattern and replaces it with a single custom hardware instruction. This reduces software overhead and demonstrates how modern AI accelerators are integrated into real processor toolchains.

The project required modifications in three major subsystems:

1. GNU Binutils (Assembler)
2. GCC Compiler
3. Spike Architectural Simulator

The entire pipeline begins from high-level C code and ends in the execution of a 32-bit custom machine opcode inside a simulated RISC-V processor.

2. Batch Normalization Equation

Batch Normalization is widely used in Machine Learning and Deep Learning models to normalize input activations and improve training stability.

The mathematical equation used in this project is:

$$y = ((x - \mu) / \sqrt{(\text{var} + \epsilon)}) \times \gamma + \beta$$

Where:

- x = input value
- μ = batch mean
- var = variance

- ϵ = small constant for numerical stability
- γ = scaling parameter
- β = shifting parameter

The compiler is designed to recognize this equation pattern directly from standard C code.

3. High-Level Working of the System

The complete system works as a multi-stage compiler and hardware execution pipeline.

Step 1 – User Writes Standard C Code

The programmer writes a normal Batch Normalization loop in C without using any custom syntax or assembly language.

Step 2 – GCC Vectorizer Detects the Pattern

Inside GCC, the custom function ``vect_recog_bnorm_pattern()'` analyzes the internal compiler representation and searches for the Batch Normalization computation pattern.

Step 3 – Internal Compiler Function Creation

Once the pattern is recognized, GCC generates an internal compiler operation called ``IFN_BNORM'`. This internal function acts as a bridge between mathematical code and hardware instructions.

Step 4 – Optab Mapping

The internal function is connected to ``bnorm_optab'` using ``optabs.def'`. The optab system maps generic compiler operations to target-specific hardware implementations.

Step 5 – RISC-V Backend Instruction Expansion

The RISC-V backend reads the optab and converts it into a hardware instruction using the machine description file ``riscv.md'`.

The final emitted assembly instruction becomes:

```
bnorm a5, a4, a3
```

Step 6 – Assembly to Machine Code

GNU Binutils converts the assembly instruction into a 32-bit machine opcode using the opcode definitions added during the project.

Step 7 – Hardware Simulation in Spike

Spike simulator executes the instruction by reading memory pointers, loading Batch Normalization parameters, computing the formula and storing results back into memory.

4. Compiler Architecture and Internal Flow

The GCC compiler internally processes the code through several layers.

4.1 GIMPLE Representation

After parsing the C code, GCC converts it into GIMPLE, a simplified intermediate representation. The Batch Normalization loop appears as arithmetic operations in this form.

4.2 Pattern Recognition

The file `'tree-vect-patterns.cc'` contains the custom function:

```
vect_recog_bnorm_pattern()
```

This function inspects mathematical operations and checks whether the structure matches the Batch Normalization formula.

4.3 Internal Compiler Functions

The file `'internal-fn.def'` defines:

```
DEF_INTERNAL_OPTAB_FN (BNORM, ECF_CONST, bnorm, binary)
```

This creates an internal function named `'IFN_BNORM'`.

4.4 Optab System

The file `'optabs.def'` contains:

```
OPTAB_D (bnorm_optab, "bnorm$I$a")
```

This connects the compiler middle-end with the target backend.

4.5 RTL Expansion

The machine description file `'riscv.md'` contains a `'define_insn'` block which instructs GCC to emit:

```
bnorm %0,%1,%2
```

during code generation.

This stage converts abstract compiler operations into actual hardware assembly instructions.

5. Assembler Modifications (GNU Binutils)

The assembler is responsible for converting assembly code into binary machine instructions.

5.1 Opcode Registration

In ``riscv-opc.h``, the following definitions were added:

```
MATCH_BNORM 0x0b  
MASK_BNORM 0xfe00707f
```

These values define the identity and structure of the custom instruction.

5.2 Instruction Registration

In ``riscv-opc.c``, the instruction was registered inside the opcode table:

```
{"bnorm", 0, INSN_CLASS_I, "d,s,t", MATCH_BNORM, MASK_BNORM, match_opcode, 0}
```

This allows the assembler and disassembler to understand the new instruction syntax.

5.3 Instruction Encoding

The instruction uses the RISC-V custom-0 opcode space. The opcode field occupies the lower 7 bits of the 32-bit instruction word.

The instruction format follows the standard R-type structure:

- opcode
- rd
- rs1
- rs2
- funct3
- funct7

This ensures compatibility with the RISC-V ISA structure.

6. Spike Simulator Execution

Since ``bnorm`` is a custom instruction, the Spike simulator must be taught how to execute it.

6.1 Instruction Registration

The instruction was registered inside ``encoding.h`` using:

```
DECLARE_INSN(bnorm, MATCH_BNORM, MASK_BNORM)
```

6.2 Instruction Execution Logic

A new file called `bnorm.h` was created inside `spike/riscv/insns/`.

This file performs the following operations:

- Reads memory addresses from registers
- Loads input arrays and parameters from RAM
- Computes Batch Normalization using floating point operations
- Stores output values back to memory
- Returns execution status to the destination register

The core formula implemented is:

$$x_hat = (x - \mu) / \sqrt{var + \epsilon}$$
$$y = \gamma \times x_hat + \beta$$

This simulates how a real hardware accelerator would operate inside a processor pipeline.

7. End-to-End Execution Pipeline

The complete execution flow of the project is:

1. User writes Batch Normalization equation in C
2. GCC converts source code into GIMPLE
3. Pattern recognition detects Batch Normalization
4. IFN_BNORM internal function is generated
5. Compiler maps operation using `bnorm_optab`
6. RISC-V backend emits `bnorm` assembly instruction
7. Assembler converts instruction into binary opcode
8. Spike simulator executes the custom instruction
9. Results are written back into memory

This demonstrates a full compiler-to-hardware execution pipeline.

8. Compilation and Testing Procedure

The project is compiled using the custom RISC-V toolchain.

Step 1 – Set Toolchain Path

```
export PATH=/home/swastik/opt/riscv/bin:$PATH
```

Step 2 – Compile the Program

```
riscv64-unknown-elf-gcc -O3 -ftree-vectorize test_di.c -o test_di -lm
```

The `-ftree-vectorize` flag activates the vectorizer pass where the custom Batch Normalization detector is implemented.

Step 3 – Verify Instruction Generation

```
riscv64-unknown-elf-objdump -d test_di
```

The disassembly output should contain the `bnorm` instruction.

Step 4 – Execute on Spike Simulator

```
spike pk test_di
```

The simulator executes the Batch Normalization operation using the custom hardware logic.

9. Importance of the Project

This project demonstrates advanced compiler engineering and hardware acceleration concepts used in modern AI processors.

The project proves that:

- Standard mathematical equations can be automatically converted into hardware instructions
- GCC can be modified to detect custom mathematical patterns
- New instructions can be integrated into the RISC-V ISA
- Hardware accelerators can be simulated using Spike
- Complete end-to-end compiler pipelines can be customized for AI workloads

The same concepts are used by companies designing AI accelerators, NPUs and custom machine learning hardware.

10. Conclusion

The project successfully implemented a complete RISC-V custom instruction pipeline for Batch Normalization acceleration. The compiler automatically detects mathematical equations written in standard C code and converts them into a dedicated hardware instruction.

The integration involved:

- Compiler modifications in GCC
- Opcode and assembler integration in GNU Binutils
- Hardware simulation logic in Spike

This project demonstrates the practical implementation of compiler optimization, ISA extension and hardware-software co-design in modern computer architecture.

Group Members

Group Number - 10

Roll No	Name	Role
24115094	Swastik Dan	Representative
24115114	Yug Rao Waghmare	Member
24115006	Aditya Boxi	Member
24115036	Devam Jignesh Parekh	Member
24115052	Koulik Poddar	Member
24115053	Kris Kumar Gupta	Member
24115093	Surendra Moond	Member
24115095	Swastik Sundar Pattjoshi	Member
24115099	Tembhurne Parag Pramod	Member
24115115	Yugantar Nag	Member